Clock gating for the cascade keyword spotter: plan and feasibility

Pedro Navarro (2132907)

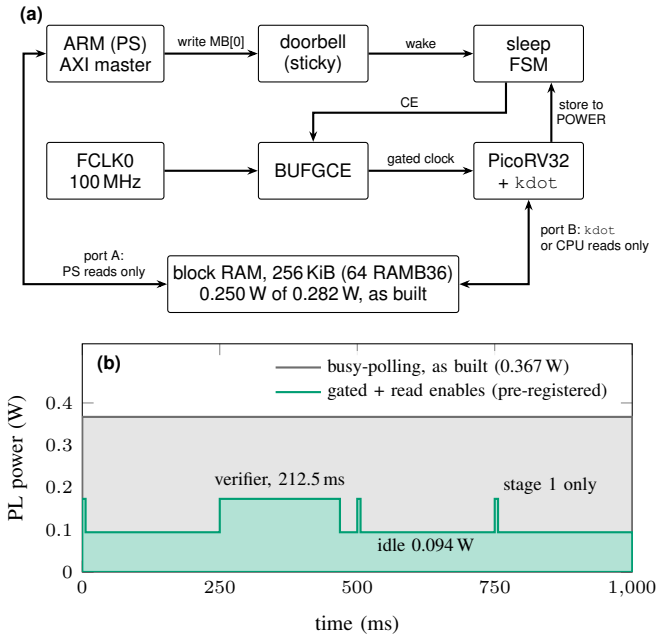


Fig. 1. (a) The idle mode R3 builds. After a request the core stores to a power register and stops; a BUFGCE then stops the clock of the core and the `kdot` unit, and the ARM's write of the mailbox word MB[0] wakes it. Read enables let each BRAM port read only when needed. (b) PL power over 1 s (four requests) for one verifier call: as built the PL draws 0.367 W all the time; with the idle mode it draws about 0.094 W between requests and about 0.173 W while awake (pre-registered estimate; verifier time from the RTL simulation). Stage-1 bursts last 0.35 ms and are drawn 6 ms wide.

Clock gating stops a circuit's clock while it has no work, so its registers stop switching and draw no dynamic power. Our keyword spotter never idles: the PicoRV32 busy-polls its mailbox and both ports of the 256 KiB block RAM read on every cycle, so the programmable logic draws 0.367 W whatever it computes. R3 adds an idle mode ported from the streaming design (Fig. 1a). The block RAM is 0.250 W of the 0.282 W dynamic power, so BRAM read enables carry most of the saving; gating the core's clock alone would save about 21 mW. Pre-registered average power for the two cascades of R2 (B1: highest recall; B2: at most 2,150 verifier calls per hour): B1 0.109 W ($-70\,\%$), B2 0.103 W ($-72\,\%$); at most 0.145 and 0.126 W if the whole core stays on while awake. Feasibility, simulated on the unchanged SoC: stage 1 takes 33,785/34,165.6/34,617 cycles (min/mean/max), equal to the board, bit-exact on 40 windows. A verifier call takes 212.5 ms, bit-exact on 3 windows, so the worst compute per request (212.9 ms) is above the 200 ms requirement.